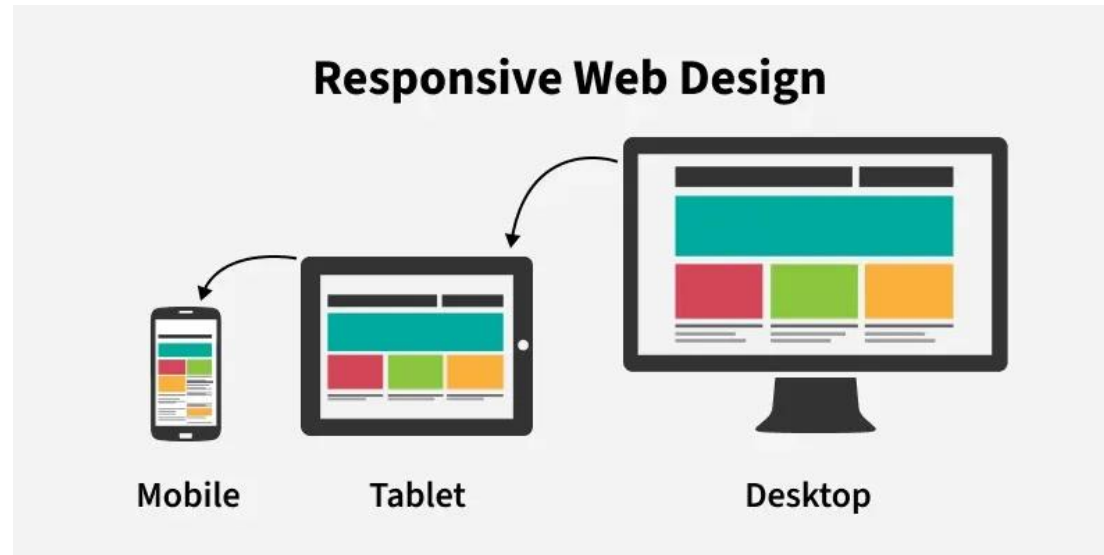


Selected Topics in HTML5 and CSS3: Responsive Web Design

Dr. Shruti bhilare

Responsive Web Design



- Responsive Web Design (RWD) is a web design and development approach in which a website automatically adapts its **layout, content, images, typography, and navigation** to provide an optimal viewing and interaction experience (UX) across different devices and screen sizes, using a **single codebase**.

Responsive Web Design: motivation

- A responsive website looks and works well on desktops, laptops, tablets, and smartphones without requiring separate versions of the site.
- Smartphones, tablets, laptops, desktops, smart TVs
 - screen size | resolution | orientation | viewport sizes
- It is the viewport according to which the webpage should adapt itself
- The layout changes based on the size and capabilities of the device.
 - For example, on a phone users would see content shown in a single column view; a tablet might show the same content in two columns.

Responsive web design

Responsive web design can be achieved using

- Meta viewport tag
- CSS media queries
- Flexible layouts
- Responsive typography
- Fluid images
- Responsive navigation

Meta viewport tag

- Strictly required for mobile-friendly responsive web design
- If not used, phone browser renders the webpage for larger “**virtual**” viewport (~980 CSS px – varies according to browser) and scale down to fit smaller screen
- Included in <head> section of your HTML.
- **You should always include this tag in your HTML.**

- `<meta name="viewport" content="width=device-width, initial-scale=1">`

- **name=viewport**: "Browser, I am going to tell you how I want the viewport to look."
- **width=device-width**: "The viewport's width should always start at the device's width." (i.e., no virtual viewport)
- **initial-scale=1**: "Start at zoom level of 100%."
- **Other properties: maximum-scale and user-scalable**

With and without viewport tag



Without the viewport meta tag



With the viewport meta tag

Media queries

- A popular technique for conditional rendering based on the characteristics of the device or browser such as screen width, height, orientation, resolution, or user preferences:
- **mediatype**: Styles can be specified for certain media types only such as for printed documents or for screen readers
 - All: Suitable for all devices.
 - Print: Intended for paged material and documents viewed on a screen in print preview mode.
 - Screen: Intended primarily for screens.
 - Speech: Intended for speech synthesizers.
- **mediafeature**: test for a specific feature of the user agent or display device.
 - Width and height: width, height, min-width, max-width, min-height, max-height
 - Orientation: landscape | portrait
 - Resolution
- For example, you can apply styles to only those screens that are greater, or smaller, than a certain width

Media queries

- `@media [media-type] and ([media-feature]) {`
 `/* CSS rules to apply */`
`}`

```
@media screen and (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Targets mobile screens

```
@media (width >= 768px) {  
  .container {  
    padding: 20px;  
  }  
}
```

Targets tablets using modern
range syntax

Mobile-first approach to web development

- Mobile First approach involves designing a desktop site starting with the mobile version, which is then adapted to larger screens contrary to the traditional approach of starting with a desktop site and then adapting it to smaller screens.
- Mobiles generally have less memory, processing power and bandwidth than other devices
- Use min-width in media-queries

Typical breakpoints for media-queries

```
/* base styles that apply to all devices */
```

```
/* Small devices (portrait tablets and large phones, 600px and up) */  
@media screen and (min-width: 600px) {...}
```

```
/* Medium devices (landscape tablets, 768px and up) */  
@media screen and (min-width: 768px) {...}
```

```
/* Large devices (laptops/desktops, 992px and up) */  
@media screen and (min-width: 992px) {...}
```

```
/* Extra large devices (large laptops and desktops, 1200px and up) */  
@media screen and (min-width: 1200px) {...}
```

Responsive or Fluid Images

- Using The width Property

If the width property is set to a percentage and the height is set to "auto", the image will be responsive and scale up and down:

```
img {  
  width: 100%;  
  height: auto;  
}
```

- Using The max-width Property

If the max-width property is set to 100%, the image will scale down if it has to, but never scale up to be larger than its original size:

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Responsive Typography: percent em,
rem, vh, vw

CSS distance units

Absolute	
2px	pixels
1mm	millimeters
2cm	centimeters
0.2in	inches
3pt	printer point 1/72 inch
Relative	
2em	2 times the element's current font size
2rem	2 times the root element's current font size
40%	40% of the parent element's size
2vh	2 times of viewport height
2vw	2 times viewport width

Whenever possible, it's best to use **relative units** like percentage, em and rem instead of absolute units (like px).

Advantage:

- Works on different screen sizes

Relative font sizes: em and rem

- Relative font sizes are usually defined in terms of em
- em represents the calculated font-size of the element
 - 1em = the inherited font size from parent element (same as font-size: 100%)
 - 2em = 2 times the inherited font size
- If you **do not** want your relative font sizes to compound through inheritance, use rem:
- rem represents the font-size of the **root element**
 - 1rem = the root (html tag) font size
 - 2rem = 2 times root font size

What is the size of ?

```
<body>
  This is
  <h1>
    <strong>120px</strong>
  </h1>
</body>
```

```
body {
  font-size: 30px;
}

strong {
  font-size: 2em;
}
```

This is

120px

```
h1 {                                user agent stylesheet
  display: block;
  font-size: 2em;
  margin-block-start: 0.67em;
  margin-block-end: 0.67em;
  margin-inline-start: 0px;
  margin-inline-end: 0px;
  font-weight: bold;
}
```

<https://codepen.io/sbhilare/pen/ExaqNxE>

- In the Chrome Inspector, we see the default browser font-size for h1 is 2em. So it's $30 * 2 * 2 = 120\text{px}$.

What is the font size of `<p>` and `<div>` elements

```
<body>
  <div>
    This is 60px
    <p>This is 22.5px</p>
  </div>
</body>
```

```
html {
  font-size: 30px;
}

div {
  font-size: 2rem;
}

p {
  font-size: .75rem;
}
```

This is 60px

This is 22.5px

- font-size is set on the html element, not body (or any other tag)

<https://codepen.io/sbhilare/pen/MWYNbeE>

Relative font

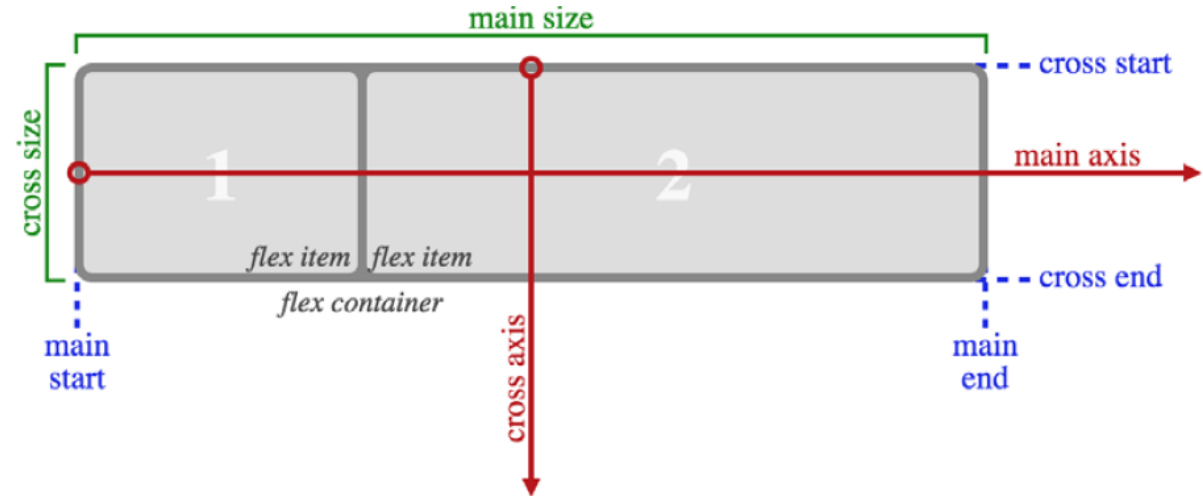
- In addition to font-size, consider em/rem for:
 - line-height
 - margin-top
 - margin-bottom

Fluid layouts

- A fluid layout is a webpage layout that automatically expands or shrinks according to the available screen or browser width, instead of using fixed dimensions.
- A fluid layout uses relative units (% , vw, Flexbox, Grid, etc.) rather than fixed pixel widths, allowing content to adapt to different screen sizes.

Flex basics

- Flex layouts are composed of:
 - **Flex container**, which contains one or more **Flex item(s)**
- You can then apply CSS properties on the **flex container** to dictate how the flex items are displayed.
- Both block and inline elements can be defined as flex containers:
 - Block container
display: flex;
 - Inline container
display: inline-flex;



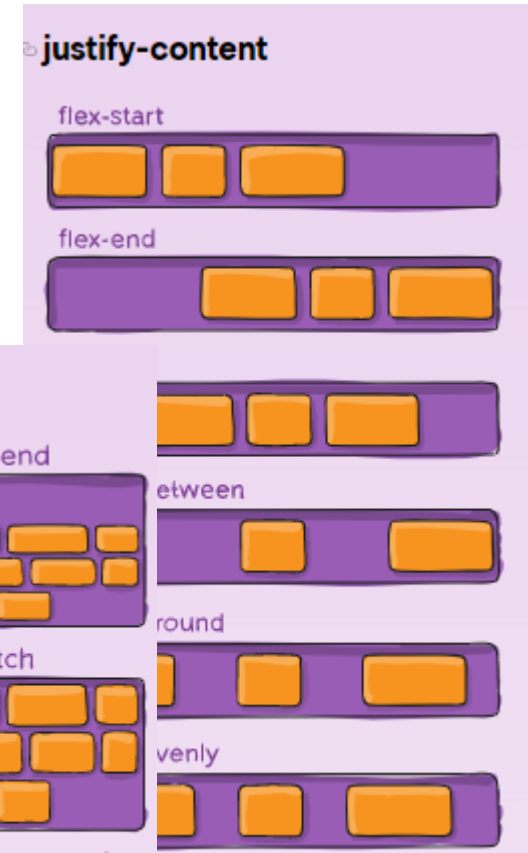
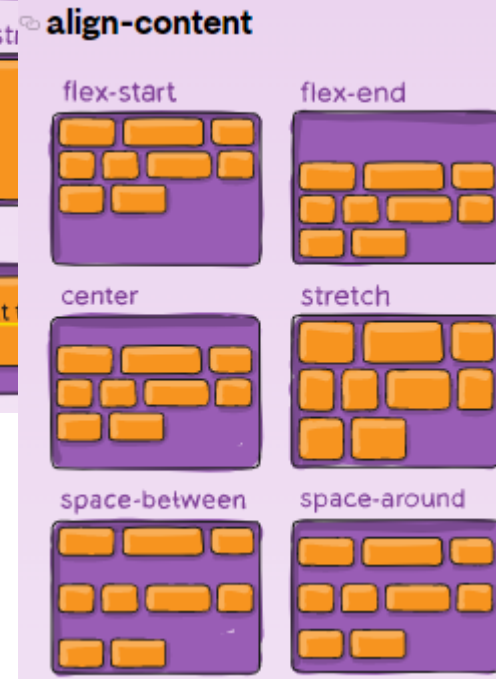
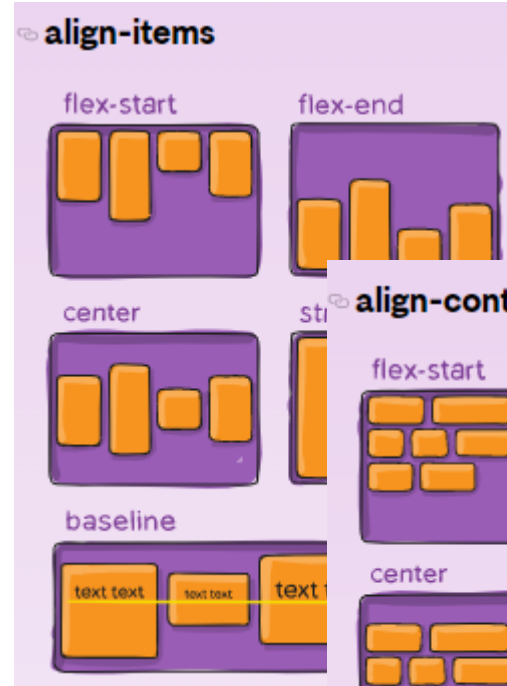
Properties on flex-container

- `display: flex | inline-flex;`
 - It enables a flex context for all its direct children.
- `flex-direction: row | column | row-reverse | column-reverse`
 - in which direction the container wants to stack the flex items
- `flex-wrap: wrap | nowrap | wrapreverse`
 - specifies whether the flex items should wrap or not.
- Flex-flow: `flex-direction* flex-wrap*`
 - shorthand property for setting both the flex-direction and flex-wrap

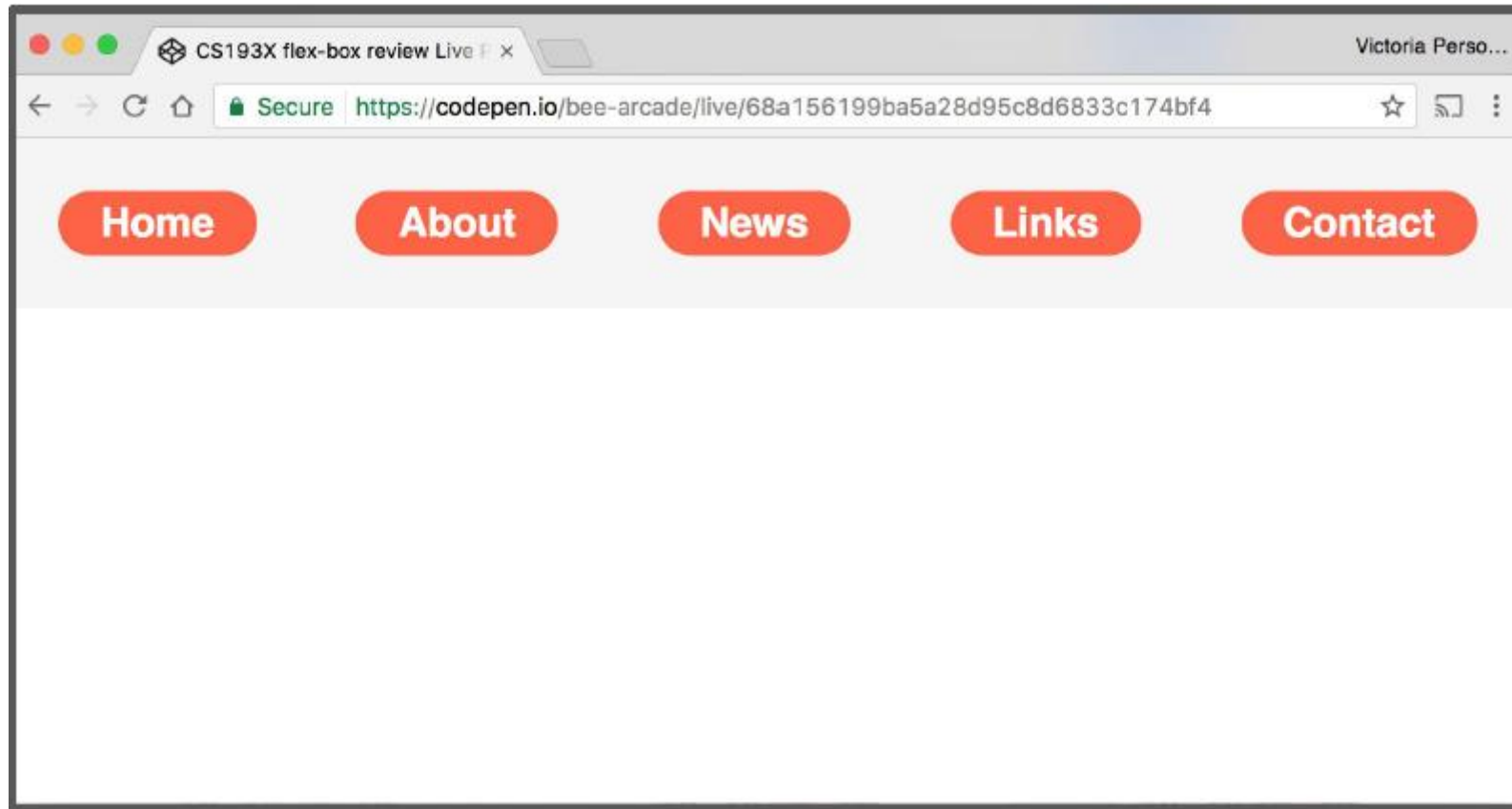
<https://codepen.io/sbhilare/pen/wvBOLJL>

Properties on flex-container

- `justify-content (main)`: `flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `space-evenly`
- `align-items(Cross)`: `stretch` | `baseline` | `flex-start` | `flex-end` | `center` | `space-between` | `space-around`
- `Align-content`: `flex-start` | `flex-end` | `center` | `space-between` | `space-around` | `stretch`
 - This aligns a flex container's lines within when there is extra space in the cross-axis, similar to how `justify-content` aligns individual items within the main-axis.
 - **Note:** This property only takes effect on multi-line flexible containers, where `flex-wrap` is set to either `wrap` or `wrap-reverse`). A single-line flexible container (i.e. where `flex-wrap` is set to its default value, `no-wrap`) will not reflect `align-content`.

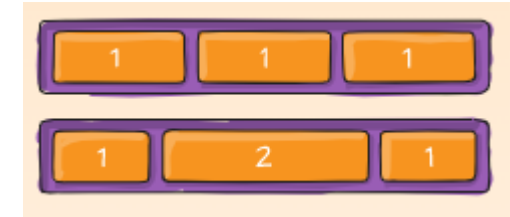


How do we create this look?



Flexbox properties on flex-items

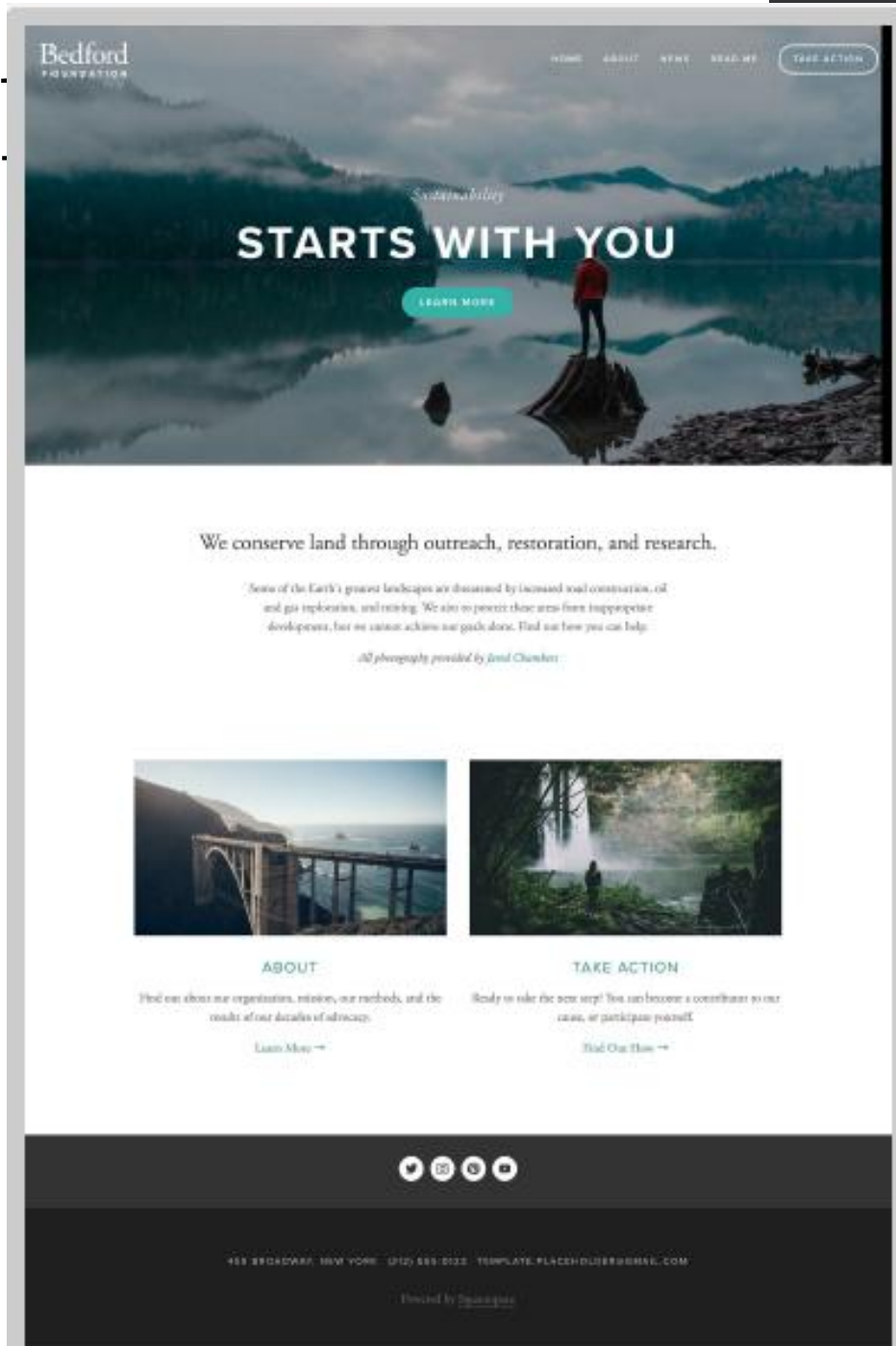
- `order:<integer>;`
 - controls the order in which they appear in the flex container.
- `flex-grow: <number>;`
 - The width of the flex item can automatically **grow larger than the flex basis** via the flex-grow property
 - If set to 1, the flex item grows itself as large as it can in the space remaining.
 - If set to 0, the flex-item does not grow.
- `flex-shrink: <number>;`
- The width of the flex item can automatically shrink **smaller than the flex basis** via the **flex-shrink** property
 - If set to 1, the flex item shrinks itself as small as it can in the space available.
 - If set to 0, the flex item does not shrink.
- `flex-basis: <length>`
- `flex`
 - This is the shorthand for **flex-grow**, **flex-shrink** and **flex-basis** combined. The second and third parameters (**flex-shrink** and **flex-basis**) are optional. The default is **0 1 auto**, but if you set it with a single number value, like **flex: 5;**, that changes the **flex-basis** to 0%, so it's like setting **flex-grow: 5; flex-shrink: 1; flex-basis: 0%**



Flex basis

- Flex items have an initial width, which, by default is either:
 - The content width, or
 - The explicitly set **width** property of the element, or
 - The explicitly set **flex-basis** property of the element
- This initial width of the flex item is called the **flex basis**.
- The explicit width of a flex item is respected *for all flex items*, regardless of whether the flex item is inline, block, or inline-block.
- If we unset the height and width, our flex items disappears, because the **flex basis** is now the content size, which is empty:

Can you create a layout like this?

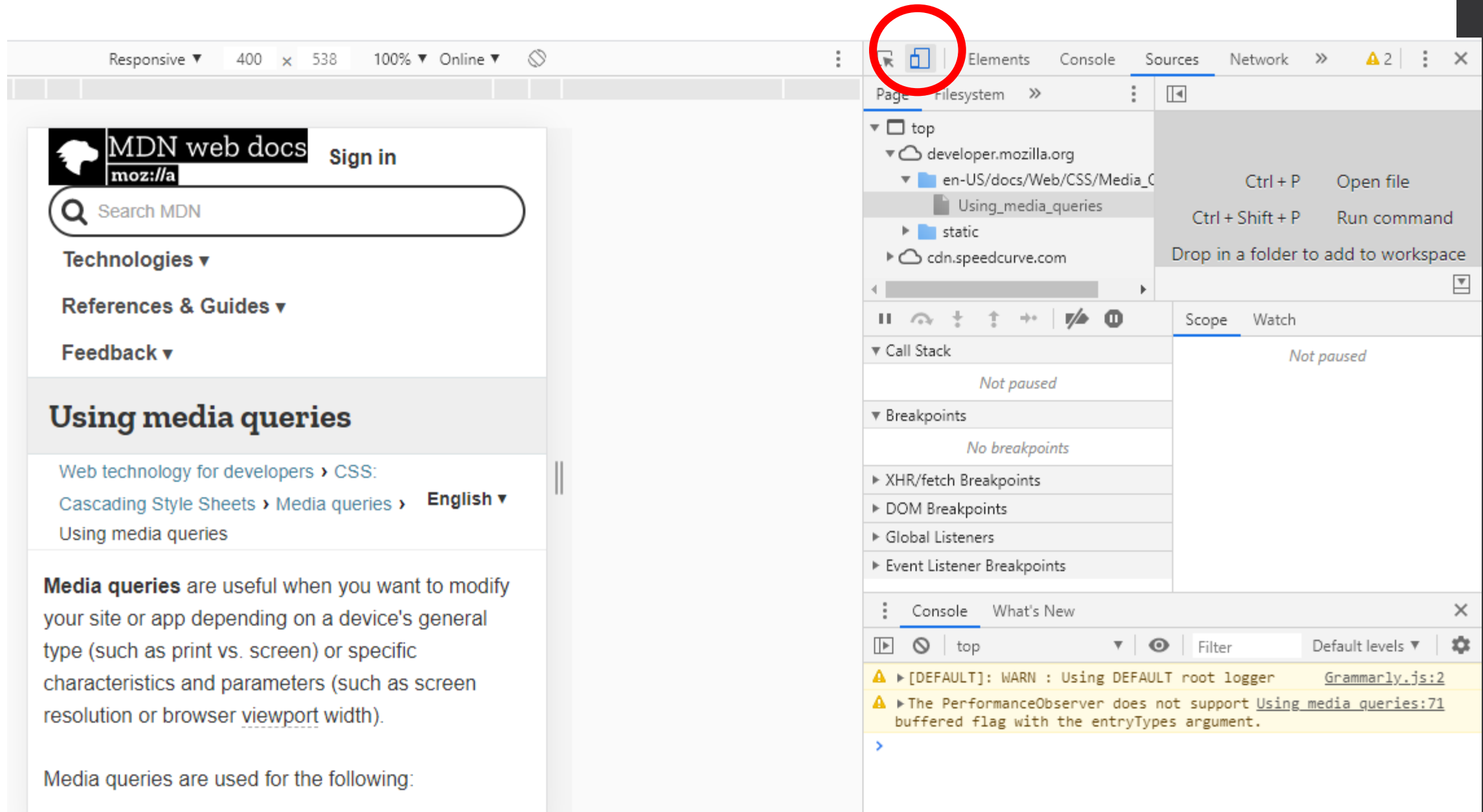


Development strategies

How do you test the mobile layouts?

- Do you upload your HTML+CSS somewhere online and navigate to that URL on your phone?
- You can simulate a web page in a mobile layout via **Chrome device mode**

Chrome device mode



The screenshot shows the Chrome DevTools interface with the 'Responsive' design tool active. The browser window displays the MDN web docs page for 'Using media queries'. The 'Sources' panel on the right is open, showing the file structure of the page. A red circle highlights the 'Device Mode' icon (a smartphone and a monitor) in the top toolbar. The 'Console' panel at the bottom shows two warnings: one about the root logger and another about the PerformanceObserver not supporting the buffered flag with the entryTypes argument.

Responsive ▾ 400 x 538 100% ▾ Online ▾

MDN web docs Sign in
moz://a

Search MDN

Technologies ▾

References & Guides ▾

Feedback ▾

Using media queries

Web technology for developers ▸ CSS:
Cascading Style Sheets ▸ Media queries ▸ English ▾
Using media queries

Media queries are useful when you want to modify your site or app depending on a device's general type (such as print vs. screen) or specific characteristics and parameters (such as screen resolution or browser viewport width).

Media queries are used for the following:

Elements Console Sources Network >> ⚠ 2

Page Filesystem >>

- top
 - developer.mozilla.org
 - en-US/docs/Web/CSS/Media_C
 - Using_media_queries
 - static
 - cdn.speedcurve.com

Ctrl + P Open file
Ctrl + Shift + P Run command
Drop in a folder to add to workspace

Scope Watch

Not paused

Not paused

Breakpoints

No breakpoints

- XHR/fetch Breakpoints
- DOM Breakpoints
- Global Listeners
- Event Listener Breakpoints

Console What's New

top Filter Default levels ⚙

- ⚠ [DEFAULT]: WARN : Using DEFAULT root logger Grammarly.js:2
- ⚠ The PerformanceObserver does not support Using_media_queries:71 buffered flag with the entryTypes argument.

Responsive design summary

- Always add the **meta viewport tag**
- Use **@media queries** to add styles for devices with certain characteristics, such as screen width
- Use flexible layouts such as flexbox and gridbox
- Use the **Chrome Device Mode** to simulate mobile rendering on desktop
- For height and width, prefer percentages
- For fonts, prefer em and rem, vh and vw
- For images, use max-width property